

Ch-1 Introduction to Software Engineering

Software

► Program - a step by step instructions to perform a specific task on a computer in a programming lang is called a prg (a set of insts).

► Software - is a program/collection of integrated prgs with proper documentation (requirement analysis, design, coding, testing) & user manuals which mainly includes installation guide & other manuals.

$S/w = prg + documentation.$

► Software components

- Prg
- Documentation - S/w documentation consists all the description, prgs, graphics & insts. pertaining to design, coding, testing & prep. of S/w.
- Operating procedure (User manual & operational manual) provides info abt what S/w is, how to work with it, how to install it on the system, how to control all the activities of the S/w.

► Software Crisis

- ① Project delays & Cost overruns - many S/w prgs exceeds their initially planned completion deadline & estimated budget significantly.
- ② Poor quality & reliability - sometimes the S/w contains numerous bugs causing reliability issues as ~~the~~ it is prone to failures & difficult to use.

(iii) Maintenance Difficulties - maintaining & updating s/w ^{sometimes becomes} extremely challenging. Changes to s/w often introduces new bugs & primes.

(iv) Lack of Proper Management ^{skilled personal} & Processes - leads to inefficient development practices

(v) Failed to meet user needs & requirements - due to significant gap b/w what user needed & what developers delivered.

(vi) Rapidly changing technology - Outstripped the ability of software development practices to keep up.

Software Engineering

SE is the systematic application of engineering principles and methods to the design, development, testing and maintenance of software products.

It involves the use of various tools, techniques, procedures, methodologies & well defined scientific principles to manage the software development process and ensure the quality, reliability & maintainability of software products.

Characteristics of software

① Intangibility - unlike physical products, s/w is intangible & doesnot have a physical form to be seen, touched. They only exist as code & functions in computer/digital devices.

② Complexity - can be extremely complex, consisting of millions of lines of codes (LOC).

③ Changeability - s/w is malleable & can be easily changed or updated to fix bugs, new features & improve performance.

④ Dependability - s/w must be reliable, ensuring to perform

as intended with correctness, consistency and must be secure, reliable &.

- ⑤ Scalability - S/w must be able to handle increasing loads, whether it's more users, data, transaction, without much degradation in performance.
- ⑥ Maintainability - S/w must be easy to maintain & update with clear documentation, & standard modular design.
- ⑦ Efficiency - should use resources (CPU, mem, net. bandwidth) efficiently to ensure optimal performance.
- ⑧ Reusability - should be reusable in diff. contexts & applications.
- ⑨ Portability - should be able to run on diff platforms & environment with minimal modifications.
- ⑩ User-friendliness - should be intuitive & easy to use, ensuring a +ve user experience.
- ⑪ No physical wear & tear - does not deteriorate overtime like physical objects.

Why should we study SE?

- i Provides solid foundation in CS principles, programming & DS.
- ii Teaches critical thinking & problem solving skills.
- iii we acquire skills of design, interface dev, testing, project management, system analysis, teamwork.
- iv Understanding S/w Lifecycles.
- v Efficient ~~under~~ scalability & performance in case of large ^{datasets}.
- vi High demand, high paying job & versatile career opportunities.
- vii Cost Reduction needed.

# Program	Software
i) Step by step insts (sets) written in a prog lang to perform a specific task. ii) Usually focused on a single specific task. iii) Typically simpler & smaller in size. iv) Components include only source code. v) May or maynot have user interface, offer very little. vi) Easier maintenance. vii) May have minimal/no documentation. viii) Author himself or small grp of specific users are target audience. ix) Eg: a calculator app	i) Collection of prgs, data & related resources that work tog to perform multiple tasks. ii) Can encompass multiple prgs & offer broader functionality. iii) Often complex, with multiple integ-rated prgs & components. Larger size. iv) Components - prgs, libraries, documentation, user interface & more. v) Typically includes a user-friendly interface. vi) Requires ongoing maintenance & updates due to complexity. vii) Comes with extensive documentation, user manuals & help files. viii) Designed for large no of users with varying tech knowledge. ix) MS Office including Excel, word, powerpoint etc.

# Hardware	Software
i) Physical components of a system such as CPU, mouse etc. ii) Cannot be easily modified. Requires physical changes/replacements to update upgrade/modify. iii) Subject to phys. wear & tear, aging & degradation.	i) Intangible set of prgs & operating info used by a system that exists in form of code & data. ii) Can be easily modified through reprogramming & upgrades. iii) Not sub to wear & tear but can become obsolete over time.

(iv) High manuf. costs & logistic expenses.

(v) Physically stored as components or devices.

(vi) Executes inst provided by S/W to perform phys. operations.

(vii) Developed through design, prototyping & manuf. process.

(viii) User interactions through i/o devices.

(ix) Errors require phy. repair/ replacement of components

(i) Less exp. to distribute & replicate after initial dev.

(v) Stored electronically in mem, cloud storage or hard drive

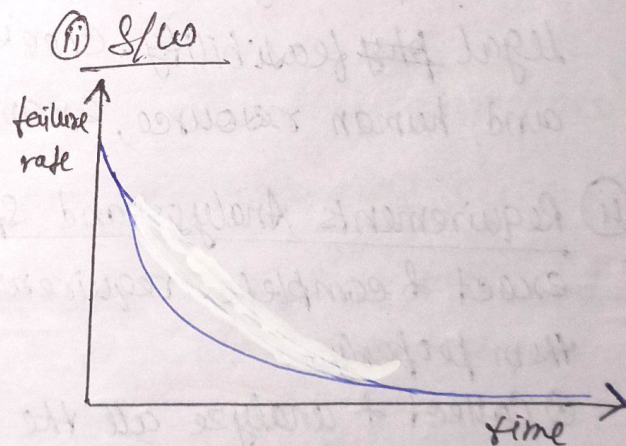
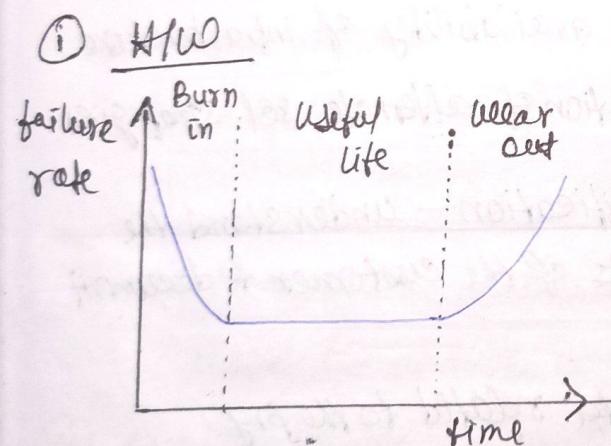
(vi) Provides inst & functionality to h/w to perform tasks.

(vii) Developed through coding, testing & debugging prog.

(viii) User interactions through GUIs & command line interfaces.

(ix) Errors require updates/patches

Failure Rates of Software & Hardware.



H/W/S/W cost



Principles of SE

- (i) Abstraction - involves hiding complex implementation details of a system & exposing only necessary & relevant parts. This helps in reducing complexity & focusing on high-level operations.
- (ii) Decomposition - involves breaking down a S/W system into distinct sections, each addressing a separate concern/aspect. Improves modularity & makes the system easier to manage.
- (iii) Encapsulation - involves bundling the data & methods that operate on the data into a single unit.

Phases of Software Development Life Cycles (SDLC)

- (i) Feasibility Analysis - determines whether the proj is viable & worth pursuing. Assess technical, economic, operational, legal ~~proj~~ feasibility. Checking availability of infrastructure and human resource, examination of alternate sol. strategies.
- (ii) Requirements Analysis and Specification - understand the exact & complete requirements of the customer & document them properly.
 - ① Collect & analyze all the data related to the proj.
 - ② Document the requirement in detail along with the functionalities of the system, called Software Requirements Specification (SRS).
 - ③ Ensure validity of each requirement as a minor error can lead to severe prob in later stages.
- (iii) Design - we transform the requirements ~~for the~~ into a blueprint structure that is suitable for implementation of the code in a specific programming lang.

① System Design - define overall architecture - system's structure, components, & their interactions.

② High Level Design - create architectural designs - data models, system interface & user interface.

③ Low Level Design - detail the internal logic, algo & ds for each component.

④ Documentation - called Software Design Description (SDD), describing how the system will perform functions.

⑤ Coding - converting the design into executable code & integrate all components to build the software system.

⑥ Combine individual components in a unified system.

⑦ We create code documentation & user guides as needed.

⑧ Testing - we need to test & verify the correct, functionality of the software, free from any bugs, errors & faults that can lead to failure.

① Unit testing - test individual components.

② Integration testing - test interaction b/w ^{integrated} components.

③ System Testing - test complete integrated system.

④ Acceptance Testing - validate the system with end-users to ensure it meets their needs & expectation.

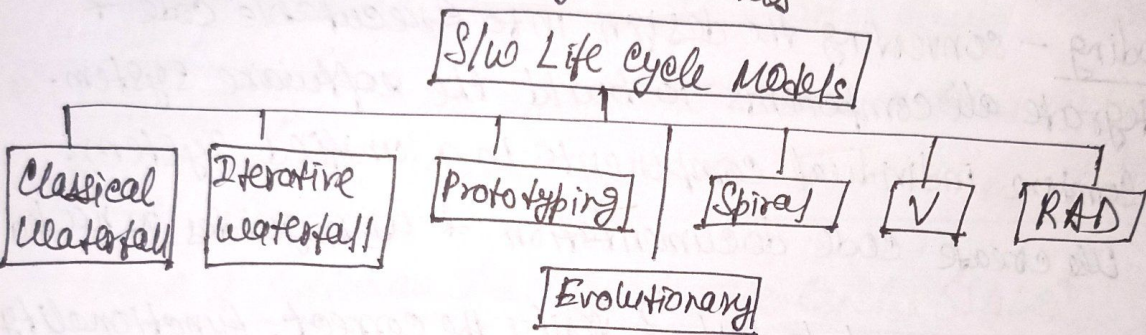
⑨ Maintenance - provide ongoing support & enhancements to ensure the s/w continues to meet user's ^{needs} & remains functional overtime. It includes - bug fixes, updates, performance monitoring.

Life Cycle Model

A S/w LCM is defined as —

- ① a descriptive & diagrammatic model of S/w Lc.
- ① identifies all activities required for prg dev.
- ① establishes a proper precedence order among diff activities
- ① divides Lc into diff phases.

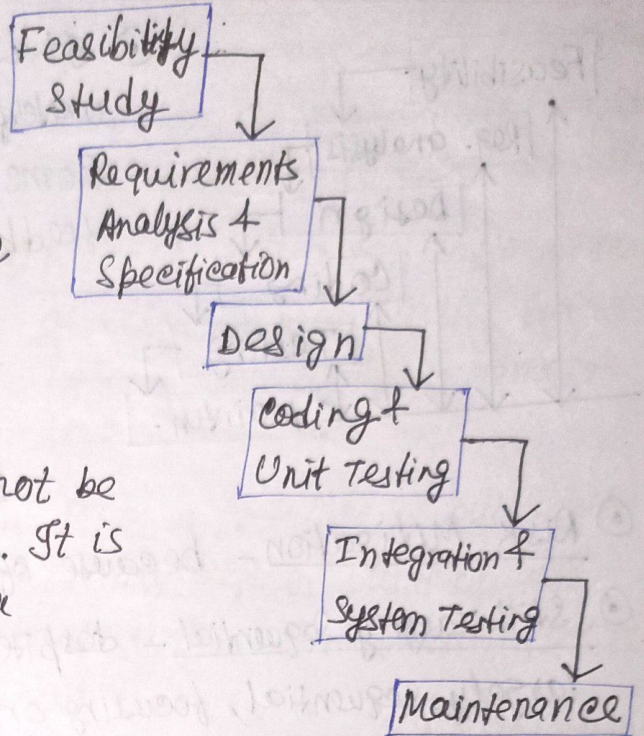
There are several life cycle models —



- ① The development team must identify a suitable LCM based on the type of S/w & always adhere to it.
- ① It helps in systematic & disciplined development of the S/w with proper entry & exit criteria for every phase.
- ① ~~while~~ During the development of the ~~prog~~ S/w, the S/w dev team often faces 99% complete syndrome. It refers to a situation where a prg appears to be nearly finished, but the few last remaining tasks take much longer than anticipated causing delays & frustration.

① Classical Waterfall Model

② It is one of the earliest & most traditional S/W dev methodologies that follows a linear & sequential approach, where each step relies on the completion of prev one.



③ Not a prac. model as it cannot be used in actual S/W dev prjs. It is generally considered to be a theoretical way of S/W dev.

④ All other LCM are essentially derived from it.

⑤ From Feasibility to Int. & System test is called development phase.

⑥ Max effort - in dev. phase is testing.

⑦ max effort - on whole is Maintenance

⑧ Called Waterfall model - as its diagrammatic repr. represents as of a waterfall.

⑨ Type of prjs

- Small to med sized prjs with clear well defined requirements
- Prjs with min changes during dev, + ^{more} predictable
- Low Risk proj.

⑩ No iteration - no going back to prev phases once completed.

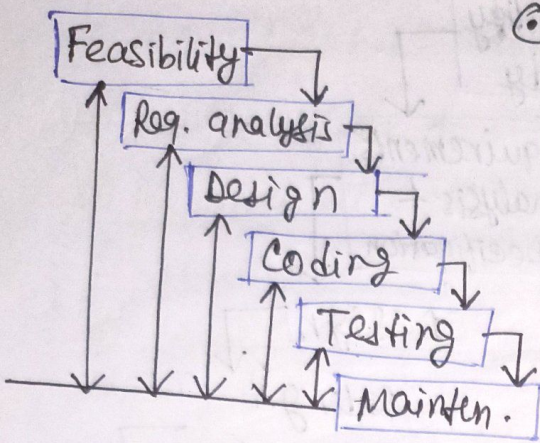
⑪ Advantages

- Simple & easy to understand
- Structured approach
- Clear doc. with well defined i/o.
- Well suited for low & fixed price contracts

⑫ Disadvantages

- Inflexibility
- High Risk & uncertainty
- Limited Customer feedback
- Not ideal for complex & uncertain prjs
- ^{med.} high cost
- High dev time.

② Iterative Waterfall Model



① It is a variation of the traditional waterfall model that incorporates some elements of iteration & feedback into the dev. process. While it still follows the sequential approach, but allows for revisiting & refining phases based on feedback/new info.

- ① Risk Mitigation - because of iteration & feedback.
- ① Still Largely Sequential - despite iter & febk, the model ^{is} still largely sequential, focusing on completing 1 phase before entering next one.
- ① Intermediate Deliverables - each stage produces deliverables that can be reviewed & refined before moving on to next phase.

① Advantages

- | | |
|----------------------------------|--|
| → Improved Flexibility | → Early error detection |
| → Structured & disciplined appo. | → More reliable & higher quality of final product. |
| → Better Risk management | → Practically Feasible |

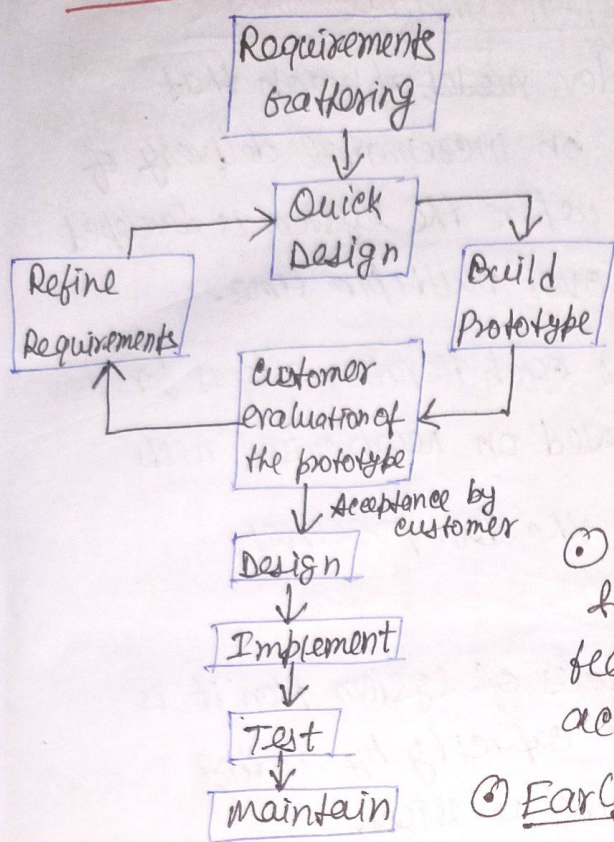
① Disadvantages

- | | |
|-----------------------------------|---|
| → Limited no of iteration | → Visiting earlier stages leads to delays |
| → Managing etc. & febk is complex | → Risk of over iteration |
| → Large T _{dev.} | |

① Types of proj

- | | |
|---|--|
| → med size proj | → Projs where most requirements are stable |
| → Projs with thorough documentation of each feedback. | |

③ Prototyping Model



① Most of the customer are not sure abt the functionality they require from the s/w, as a result the final s/w is not according to exact demand.

② Prototype - is a key implementation of a system with limited functionalities, inefficient performance, & low reliability.

③ In PTM, a prototype is built, tested & iteratively refined based on user feedback until a satisfactory sol. is achieved.

④ Early user involvement - to provide feedback

⑤ Quick & Incomplete versions, focusing on only key features & func.

⑥ Exploratory & Experimental - allowing developers to try diff ideas & approaches to find the best sol.

⑦ Focuses on refining & clarifying requirements.

⑧ Disposal or Evolutionary prototypes.

⑨ Advantages

- Better requi. understanding
- Reduced Risk
- Improved Communication
- More likely to meet user expectation due to feedback nature
- Flexibility & Adaptability

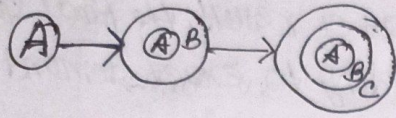
⑩ Disadvantages

- High Tdev & Cost dev
- High complexity
- Due to quick builds, prototypes lack perf. structure, scalability.
- If user is not satisfied with proto. he may lose interest in the proj

⑪ Types of proj.

- Unclear & evolving requi.
- Proj with high risk & uncn.
- User centered proj
- Research & dev. proj.

④ Evolutionary / Successive Versions / Incremental Model (with iteration)



① It is a ^{s/w} dev. ~~model~~ approach that emphasises on incremental delivery of a system, where the system is developed & refined over multiple time.

② Flexibility in Requirements - as each iteration allows for adjustment & enhancements based on new/revised needs.

③ User involvement, throughout the dev. processes.

④ Less risks & uncertainty

⑤ First we develop the core modules of system then it is refined into increasing lvs of capacity by adding new functionalities in successive version.

⑥ Advantages

- | | |
|--------------------|--------------------------------|
| → Flexibility | → Improved risks |
| → User involvement | → Enhance quality. |
| | → useful for larger size prjms |

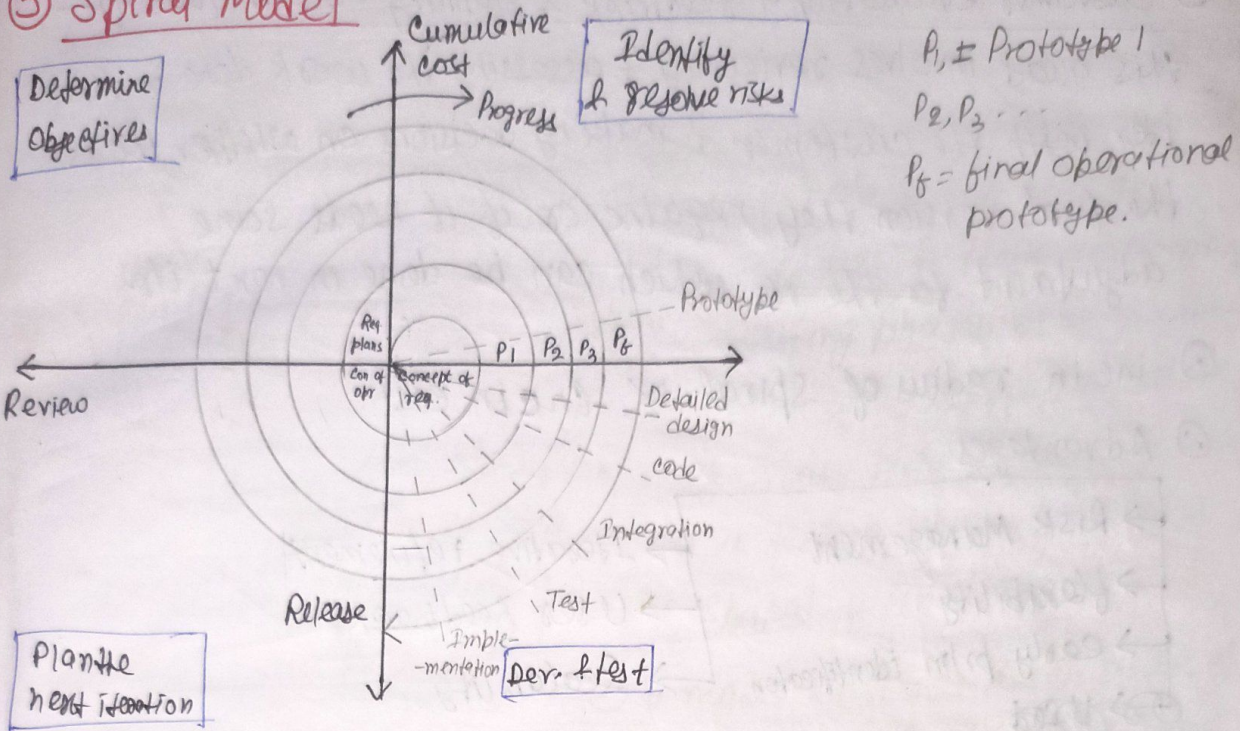
⑦ Disadvantages

- | | |
|--------------------------|-----------------------------|
| → Complex prj management | → T _{dev} is large |
| → med cost | |

⑧ Types of prj

- | | |
|-----------------------------------|-----------------------|
| → Prjs with Evolving req. | → Large/Complex prjms |
| → High-risk prjs | → User centered prjs |
| → Innovation & experimental prjs. | |

⑤ Spiral Model



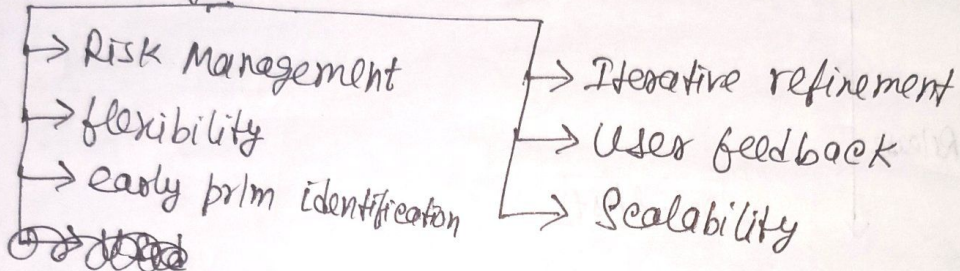
- ① It combines elements of both iterative development & the Waterfall model and also emphasizes on risk management & iterative refinement of a prj through multiple spirals.
- ② Each spiral represents a phase of development of a prototype including planning/Determine objectives (Quad 1), risk analysis & management (Quad 2), Development & testing (Quad 3) and Customer evaluation & Feedback (Quad 4).
- ③ Objective Setting/Planning (Quad 1) - this quad focuses on determining the objectives, goals, resource allocation, limitations regarding the prj, identifying alternatives. This results in clear objectives, possible approaches & resource planning.
- ④ Identifying & managing risks (Quad 2) - this quad focuses on identifying & analyzing potential risks that could impact the prj. It also aims for ^{determining} risk mitigation strategies.
- ⑤ Development & Testing (Quad 3) - this quad is the actual dev & testing phase, where after risk analysis & mitigation strategies developers put all the designing & coding efforts and then ~~it~~ integrate, test & implement the ^{new} ~~final~~ version.

⑥ Customer evaluation & feedback & planning next itr (quad 4) -

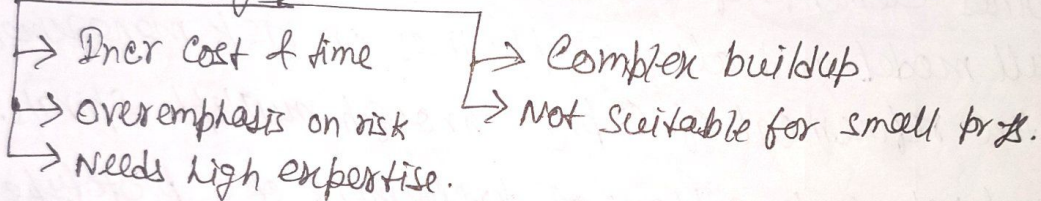
this quad involves reviewing & assessing the work done in this iter with the customer & making decisions on whether this is the final version they require or if it needs some adjustment ~~for the~~ which can be done in next itr.

⑦ inc in radius of spiral $\propto$ inc in cost.

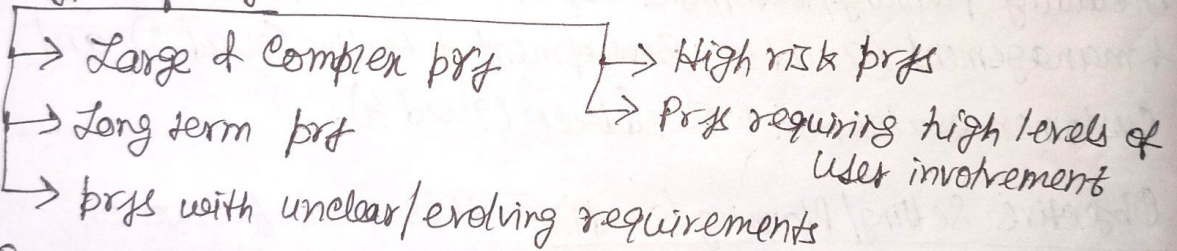
⑧ Advantages



⑨ Disadvantages



⑩ Types of Prj



⑪ Spiral Model as a Meta Model - SM is considered as a meta model because it incorporates elements from various other traditional s/w dev. models such as waterfall, iterative prototyping, evolutionary. A meta model is essentially a model that provides a framework or structure for other models.

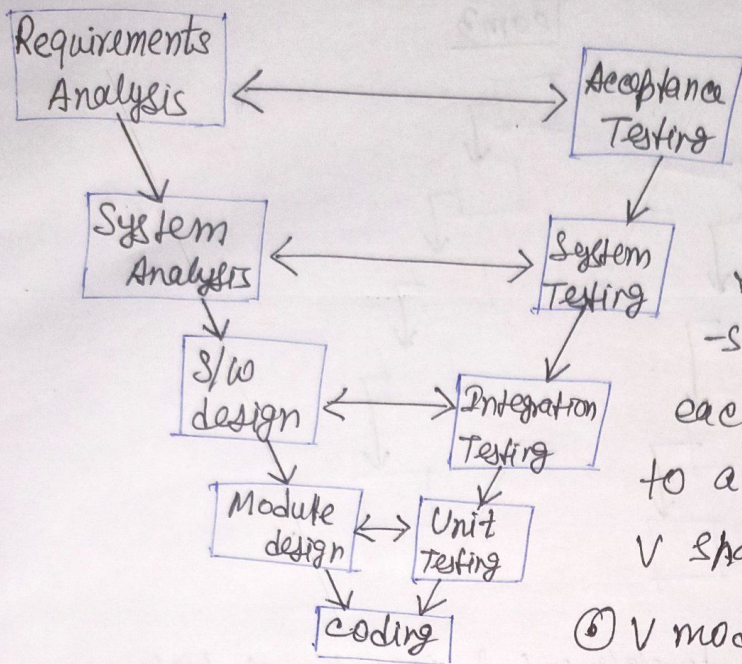
⑫ Structured ^{sequential} approach like waterfall model

⑬ iterative nature like Iterative W.M.

⑭ Focus on prototypes like PM

⑮ Uses a evolutionary approach through iteration like EM

⑥ V-Model (Verification & Validation)



⑥ It is a s/w dev process that emphasizes a parallel relationship btw dev phases & their corresponding testing phases. It is an extension of Waterfall M, where each dev phase is directly linked to a testing phase forming a V shaped when diagrammed.

⑥ V model ensures each phase of dev. is verified & validated before moving to the next phase.

⑥ Sequential process similar to WEM.

⑥ Rigid model - does not easily accommodate changes once dev. process started

⑥ Advantage

- Easy to Understand & use
- High quality product
- Early defect detection
- Clear, well defined stages with disciplined approach

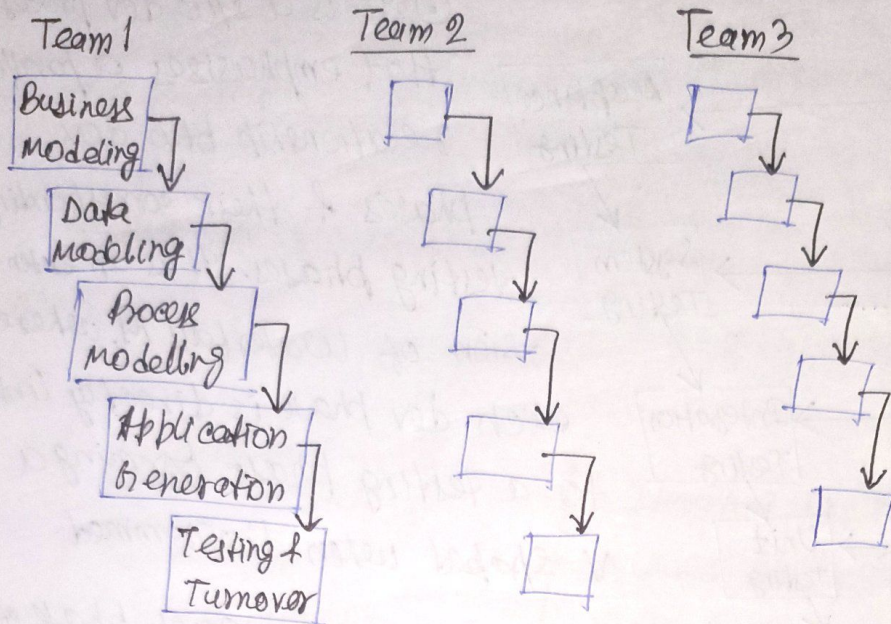
⑥ Disadvantages

- Inflexible
- Risk of unclear requirement
- Not suitable for complex & long term proj.

⑥ Types of Proj

- Well-Defined Requirements
- Proj with clear deliverable
- Small - med sized proj.

⑦ RAD (Rapid Application Development) Model



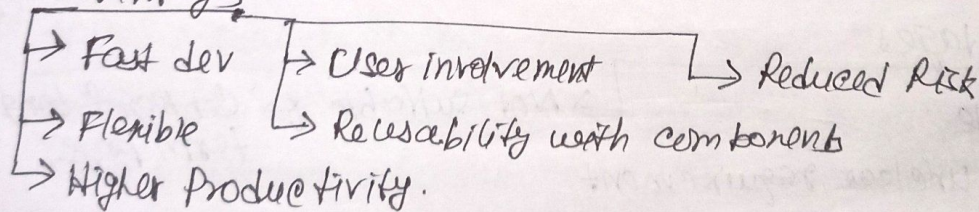
① It emphasizes on quick development & iteration of prototypes, rather than extensive upfront planning.

② It is a high speed adaptation of the linear sequential model in which rapid dev. is achieved by using component based construction along with reusability & parallel dev of components.

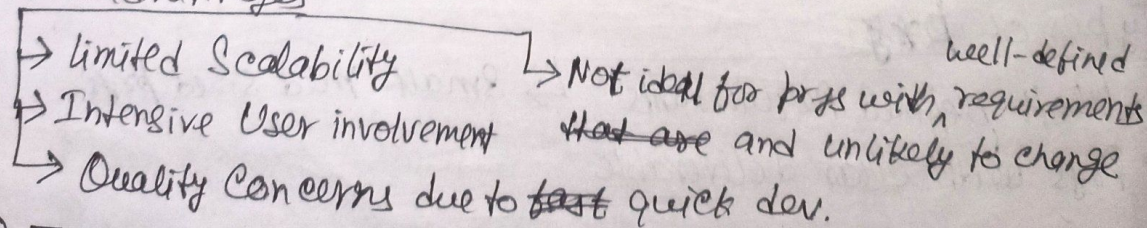
③ User ~~develop~~ involvement throughout the dev.

④ Highly flexible & adaptive.

⑤ Advantages



⑥ Disadvantages



⑦ Types of prot.

